

IF332

FORMAL LANGUAGES

&

AUTOMATA THEORY

09
DERIVATION TREE

DENNIS GUNAWAN

REVIEW

Production Rules of FSA:

Regular Grammar

Conversion of FSA to Regular Grammar

Conversion of Regular Grammar to FSA

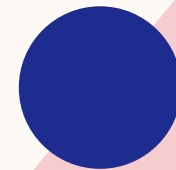
OUTLINE

Context Free Grammar

Derivations from a Grammar

Parsing

Ambiguous Grammar



CONTEXT FREE GRAMMAR

- Context Free Grammar is defined by 4 tuples as $G = (V, T, P, S)$
 - V : set of variables or non-terminal symbols
 - T : set of terminal symbols
 - P : production rules for terminals and non-terminals
 - S : start symbol
- Context Free Grammar has production rule of the form $A \rightarrow \alpha$ where $\alpha \in (V \cup T)^*$ and $A \in V$
- Examples
 - $B \rightarrow CDeFg$
 - $D \rightarrow BcDe$

DERIVATIONS FROM A GRAMMAR

- The set of all strings that can be derived from a grammar is said to be the LANGUAGE generated from that grammar

- $G_1 = (\{S, A, B\}, \{a, b\}, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}, S)$

$$S \Rightarrow AB \Rightarrow ab$$

$$L(G_1) = \{ab\}$$

DERIVATIONS FROM A GRAMMAR

■ $G_2 = (\{S, A\}, \{a, b\}, \{S \rightarrow aAb, A \rightarrow aAb \mid \varepsilon\}, S)$

$$S \Rightarrow aAb \Rightarrow ab$$

$$S \Rightarrow aAb \Rightarrow aaAbb \Rightarrow aabb$$

$$S \Rightarrow aAb \Rightarrow aaAbb \Rightarrow aaaAbbb \Rightarrow aaabbb$$

$$L(G_2) = \{a^n b^n \mid n \geq 1\}$$

DERIVATIONS FROM A GRAMMAR

■ $G_3 = (\{S, A, B\}, \{a, b\}, \{S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bB \mid b\}, S)$

$$S \Rightarrow AB \Rightarrow ab$$

$$S \Rightarrow AB \Rightarrow aAbB \Rightarrow aabb$$

$$S \Rightarrow AB \Rightarrow aAb \Rightarrow aab$$

$$S \Rightarrow AB \Rightarrow abB \Rightarrow abb$$

$$L(G_3) = \{ab, a^2b^2, a^2b, ab^2, \dots\} = \{a^m b^n \mid m \geq 1 \text{ and } n \geq 1\}$$

DERIVATIONS FROM A GRAMMAR

- Method to find whether a string belongs to a grammar or not
 1. Start with the start symbol and choose the closest production that matches to the given string.
 2. Replace the variables with its most appropriate production. Repeat the process until the string is generated or until no other productions are left.

DERIVATIONS FROM A GRAMMAR

- Verify whether the grammar

$$S \rightarrow 0B \mid 1A$$

$$A \rightarrow 0 \mid 0S \mid 1AA \mid \varepsilon$$

$$B \rightarrow 1 \mid 1S \mid 0BB$$

generates the string 00110101.

S	
$\Rightarrow 0B$	$(S \rightarrow 0B)$
$\Rightarrow 00BB$	$(B \rightarrow 0BB)$
$\Rightarrow 001B$	$(B \rightarrow 1)$
$\Rightarrow 0011S$	$(B \rightarrow 1S)$
$\Rightarrow 00110B$	$(S \rightarrow 0B)$
$\Rightarrow 001101S$	$(B \rightarrow 1S)$
$\Rightarrow 0011010B$	$(S \rightarrow 0B)$
$\Rightarrow 00110101$	$(B \rightarrow 1)$

DERIVATIONS FROM A GRAMMAR

- Verify whether the grammar

$$\begin{aligned} S &\rightarrow aAb \\ A &\rightarrow aAb \mid \varepsilon \end{aligned}$$

generates the string *aabbb*.

$$\begin{aligned} S & \\ \Rightarrow aAb & \quad (S \rightarrow aAb) \\ \Rightarrow aaAbb & \quad (A \rightarrow aAb) \\ \Rightarrow aabb & \quad (A \rightarrow \varepsilon) \end{aligned}$$

$$\begin{aligned} S & \\ \Rightarrow aAb & \quad (S \rightarrow aAb) \\ \Rightarrow aaAbb & \quad (A \rightarrow aAb) \\ \Rightarrow aaaAbbb & \quad (A \rightarrow aAb) \\ \Rightarrow aaabbb & \quad (A \rightarrow \varepsilon) \end{aligned}$$

DERIVATION TREE

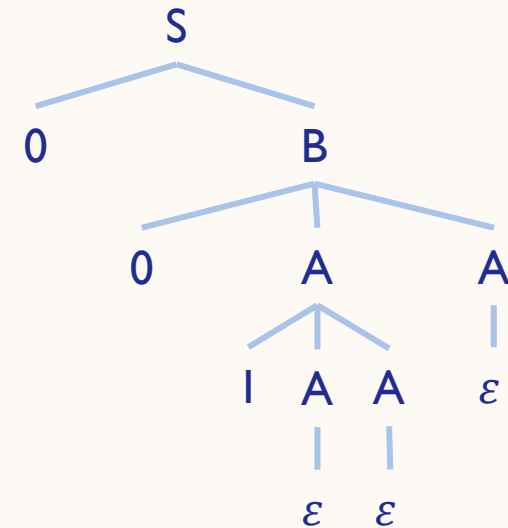
- A Derivation Tree or Parse Tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a Context Free Grammar

- Example

$$S \rightarrow 0B$$

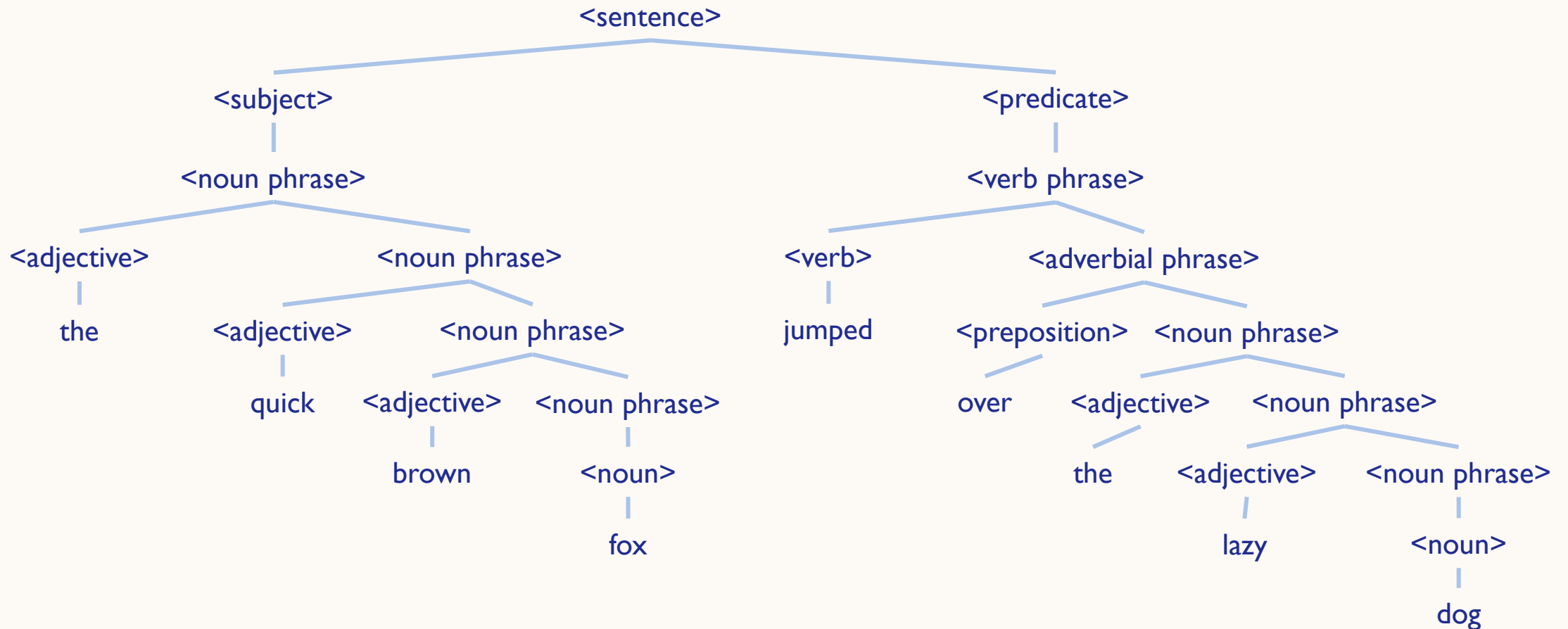
$$A \rightarrow 1AA \mid \varepsilon$$

$$B \rightarrow 0AA$$



EXAMPLE

THE QUICK BROWN FOX JUMPED OVER THE LAZY DOG



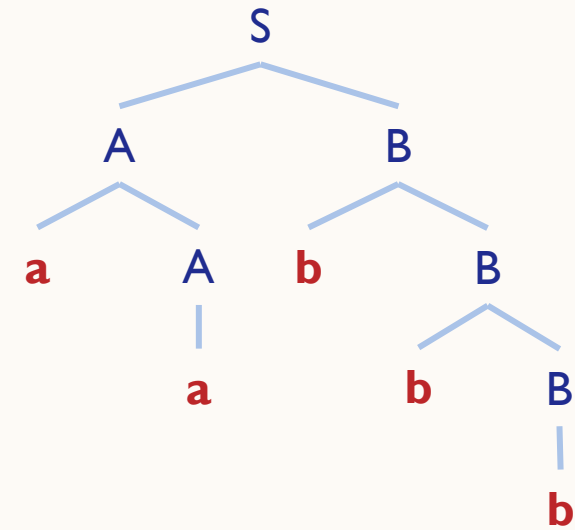
PARSING

Construct a Decision Tree for generating the string *aabbb* from the grammar

$$S \rightarrow AB$$

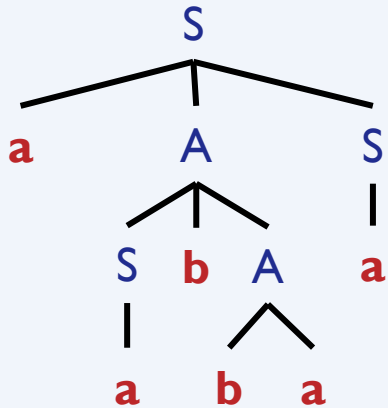
$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$



PARSING

Leftmost Derivation	Rightmost Derivation
Applying production to the leftmost variable	Applying production to the rightmost variable
Generating the string <i>aabbbaa</i> from the grammar $S \rightarrow aAS \mid a$ $A \rightarrow SbA \mid ba$	
$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS$ $\Rightarrow aabbaS \Rightarrow aabbbaa$	$S \Rightarrow aAS \Rightarrow aAa \Rightarrow aSbAa$ $\Rightarrow aSbbbaa \Rightarrow aabbbaa$



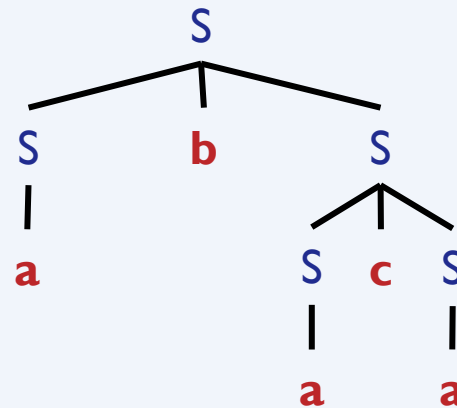
PARSING

Left Derivation Tree	Right Derivation Tree
A Left Derivation Tree is obtained by applying production to the leftmost variable in each step	A Right Derivation Tree is obtained by applying production to the rightmost variable in each step
Generating the string <i>aabaa</i> from the grammar $S \rightarrow aAS \mid aSS \mid \varepsilon$ $A \rightarrow SbA \mid ba$	
<pre>graph TD S1[S] --- a1[a] S1 --- S2[S] S1 --- S3[S] S2 --- a2[a] S2 --- A[A] S2 --- S4[S] A --- b[b] A --- a3[a] S4 --- a4[a] S4 --- S5[S] S4 --- S6[S] S5 --- e1[ε] S6 --- e2[ε]</pre>	<pre>graph TD S1[S] --- a1[a] S1 --- S2[S] S1 --- S3[S] S2 --- e1[ε] S3 --- a2[a] S3 --- A[A] S3 --- S4[S] A --- b[b] A --- a3[a] S4 --- a4[a] S4 --- S5[S] S4 --- S6[S] S5 --- e2[ε] S6 --- e3[ε]</pre>

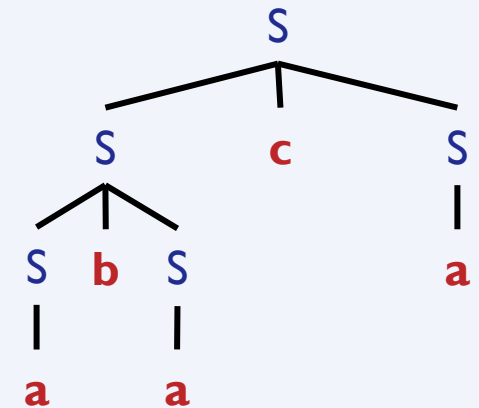
AMBIGUOUS GRAMMAR

- A grammar is said to be ambiguous if there exists more than one derivation trees for a string in T^* (more than one Leftmost Derivation Trees or more than one Rightmost Derivation Trees)
- Generating the string *abaca* from the grammar $S \rightarrow SbS \mid ScS \mid a$

$S \Rightarrow SbS \Rightarrow abS \Rightarrow abScS \Rightarrow abacS \Rightarrow abaca$



$S \Rightarrow ScS \Rightarrow SbScS \Rightarrow abScS \Rightarrow abacS \Rightarrow abaca$



CONVERSION OF REGULAR EXPRESSION TO CFG

Regular Expression	Context Free Grammar
$RE = \varepsilon$	$S \rightarrow \varepsilon$
$RE = \emptyset$	
$RE = a$	$S \rightarrow a$
$RE = a + b$	$S \rightarrow a \mid b$
$RE = ab$	$S \rightarrow ab$
$RE = a^*$	$S \rightarrow aS \mid \varepsilon$

CONVERSION OF REGULAR EXPRESSION TO CFG

Convert the regular expression $(ab + ba)^*(abb)^*$ to CFG.

$$S \rightarrow AB$$

$$A \rightarrow CA \mid \varepsilon$$

$$C \rightarrow ab \mid ba$$

$$B \rightarrow DB \mid \varepsilon$$

$$D \rightarrow abb$$

$$\text{CFG } G = (\{S, A, B, C, D\}, \{a, b\}, P, S)$$



PRACTICE

EXERCISES

1. Construct a Decision Tree for generating the string *bbabaaba* from the grammar below.

$$S \rightarrow AA$$

$$A \rightarrow AAA \mid a \mid bA \mid Ab$$

2. Construct a Decision Tree for generating the string *accd* from the grammar below.

$$S \rightarrow aAd \mid aB$$

$$A \rightarrow b \mid c$$

$$B \rightarrow ccd \mid ddc$$

EXERCISES

3. Generate the string *baabaab* from the grammar below.

$$S \rightarrow AB$$

$$A \rightarrow Aa \mid bB$$

$$B \rightarrow a \mid Sb$$

4. Generate the string *bbaaaabb* from the grammar below.

$$S \rightarrow Ba \mid Ab$$

$$A \rightarrow Sa \mid AAb \mid a$$

$$B \rightarrow Sb \mid BBa \mid b$$

EXERCISES

5. The following grammar generates the language of regular expression $0^*1(0 + 1)^*$.

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \varepsilon$$

$$B \rightarrow 0B \mid 1B \mid \varepsilon$$

Give leftmost & rightmost derivations and parse trees of the following strings.

- a) 00101
- b) 1001
- c) 00011

EXERCISES

6. Consider the grammar below.

$$S \rightarrow aS \mid aSbS \mid \varepsilon$$

This grammar is ambiguous. Show in particular that the string *aab* has two:

- a) parse trees
- b) leftmost derivations
- c) rightmost derivations

EXERCISES

7. Prove that the grammar below is ambiguous by generating the string $a + a * b$.

$$S \rightarrow S + S \mid S * S \mid a \mid b$$

8. The grammar below is ambiguous. Show that the string $aabbccdd$ has two leftmost derivations and parse trees.

$$S \rightarrow AB \mid C$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cBd \mid cd$$

$$C \rightarrow aCd \mid aDd$$

$$D \rightarrow bDc \mid bc$$

EXERCISES

9. Prove that the grammar below is ambiguous.

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA$$

$$B \rightarrow b \mid bS \mid aBB$$

REFERENCES

- Utdirartatmo, F. (2005), Teori Bahasa dan Otomata (edisi ke-2), Graha Ilmu.
- Hopcroft, John E., Rajeev Motwani, and Jeffrey D. Ullman (2006), Introduction to Automata Theory, Languages, and Computation (3rd edition), Addison-Wesley.

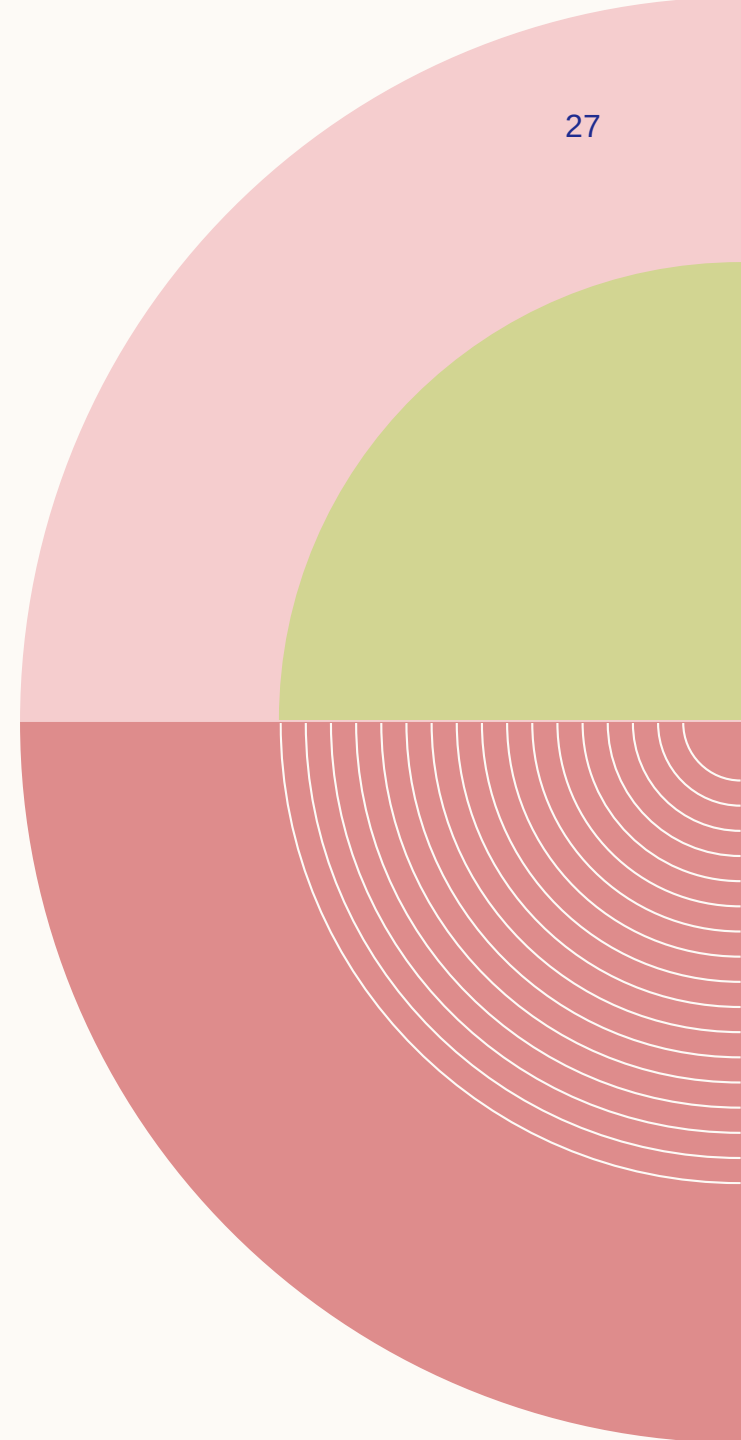
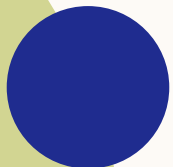
NEXT

Simplification of Context Free Grammar (CFG):

Elimination of Useless Productions

Elimination of Unit Productions

Elimination of Null (ϵ) Productions



VISION

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.

MISSION

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology



iF INFORMATIKA
UMN